

Digital Image



Bitmap (raster) vs. Vector graphics

Two possible ways to represent graphic information

- *Bitmap (or raster)*
- *Vector*

A bitmap image is composed of a set of “dots” on a grid

GIF, JPEG, PNG, BMP, TIFF, etc., are all bitmap formats

A vector image is *described mathematically*

Through parameters, coordinates, equations, etc.

Bitmap graphics

Information about the color of each pixel is stored in the image file

- Suitable for photographs and paintings
- Pictures acquired through a scanner or a digital camera are bitmap images



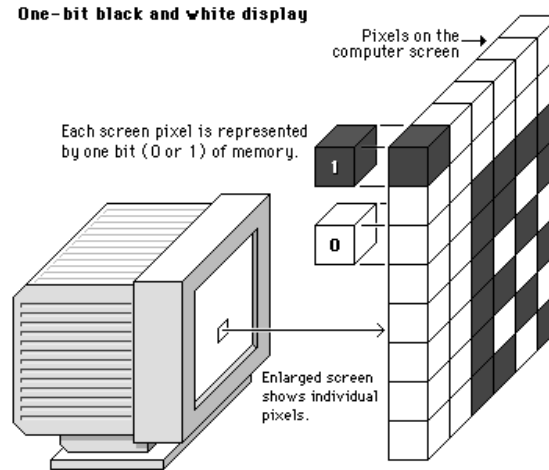
Pixels and colors...

The computer's operating system treats the screen as a grid of *pixels* (*picture elements*)

Each pixel has an associated amount of video memory, called *bit depth*

E.g., screen with on/off pixels

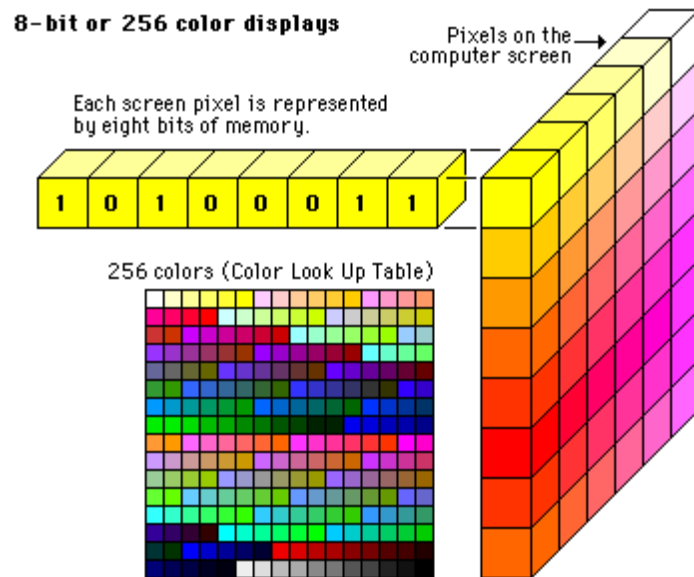
1 bit per pixel (0/1)



... pixels and colors...

More video memory → more colors

- E.g., 8 bits per pixel → 256 ($= 2^8$) colors
 - Enough if image quality is not an issue
 - Color space *quantized* into 256 possible levels
- Using 16 bits per pixel → 65,536 ($= 2^{16}$) colors
 - Color space quantized into 65,536 possible levels



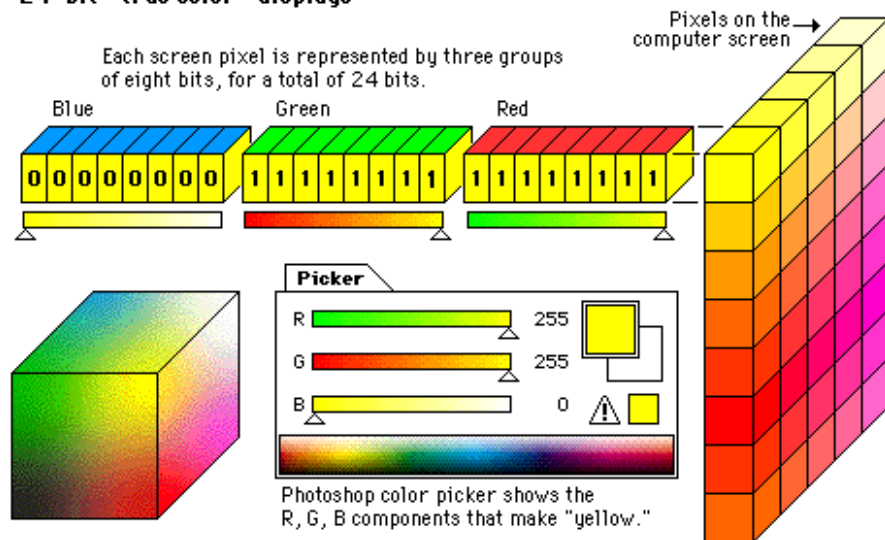
... pixels and colors

True color when each pixel has at least 24 bits associated with it

- Millions of colors
- 8 bits for *Red*
- 8 bits for *Green*
- 8 bits for *Blue*

Color space
quantized into
16,777,216 ($=2^{24}$)
levels

24-bit “true color” displays



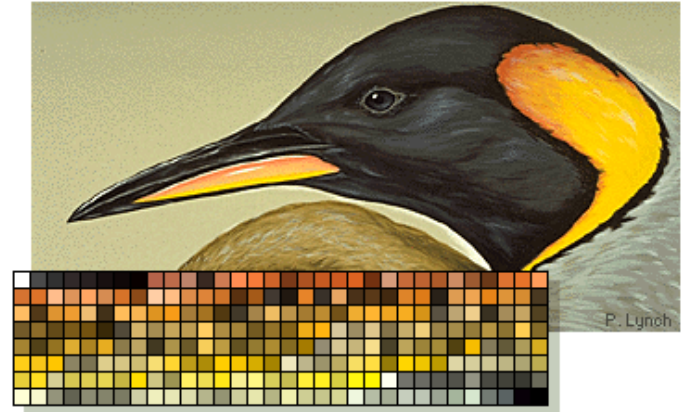
Images and colors...

Considerations similar to those about screen pixels

E.g., 256-color images use 8 bits for each pixel

The 256 colors are defined within
a *palette*, called CLUT (Color LookUp Table)

Important: different images can use
different sets of 256 colors



Images using 16 bits per pixel (i.e., $2^{16} = 65,536$ colors) also need a CLUT

Bitmap graphics – “True color” images

... images and colors

True-color images

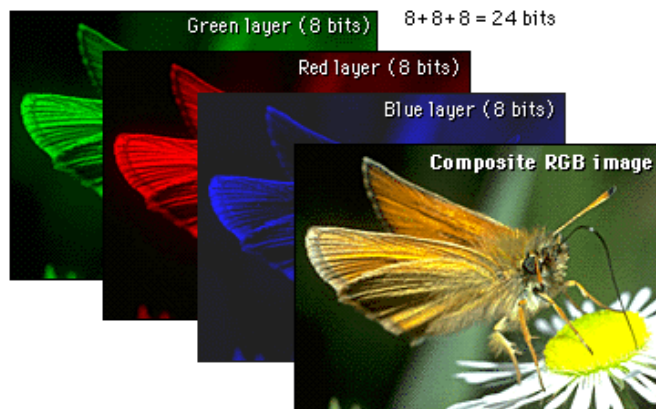
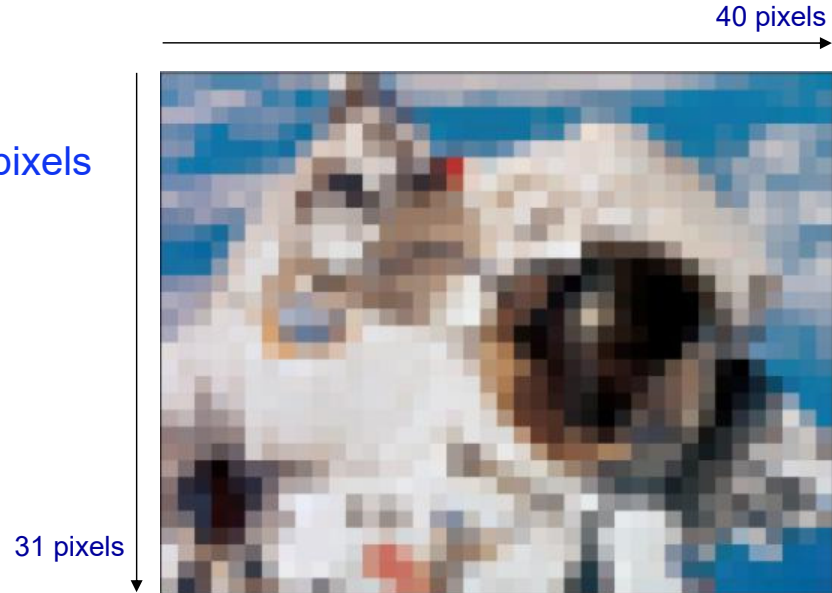


Image size

Image size is the *spatial* size of the image

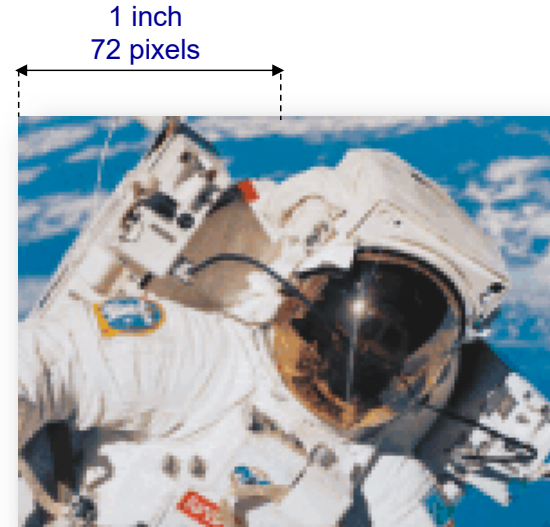
- Width x height
- Measured in pixels, cm, inches, ...
- The bigger the image, the more the pixels



Spatial resolution...

Spatial resolution is the number of pixels/dots per length unit

- Relationship between image size and number of pixels/dots composing the image
- Measured in *dots per inch* (dpi) or *pixels per inch* (ppi)



- Necessary to “foresee” the size (cm, inches) of a printed image
- Necessary to “foresee” the size (pixels) of an image acquired through a scanner

... spatial resolution...

Example 1

Image 206 x 301 pixels, resolution 72 ppi

- Printed size: width = $206/72'' = 2.861''$, height = $301/72'' = 4.181''$
- Maintaining the size in pixels constant and doubling the resolution (144 dpi), the width of the printed image will be $206/144'' = 1.431''$, and the height $301/144'' = 2.09''$ (smaller but “crisper”)
- Maintaining the size in pixels constant and halving the resolution (36 dpi), the width of the printed image will be $206/36'' = 5.722''$, and the height $301/36'' = 8.361''$ (bigger but coarser)
- Maintaining the size in inches constant and doubling the resolution (144 dpi), the width in pixels will be $206 \cdot 2 = 412$ and the height $301 \cdot 2 = 602$ (new pixels are added)
- Maintaining the size in inches constant and halving the resolution (36 dpi), the width in pixels will be $206/2 = 103$ and the height $301/2 = 151$ (half of the pixels are removed)

... spatial resolution

Example 2

Image 2.861 x 4.181 inches acquired through a scanner

- With a 72 dpi scanning resolution, the obtained image will be $2.861 \cdot 72 = 206$ pixels wide and $4.181 \cdot 72 = 301$ pixels high
- With a 144 dpi scanning resolution, the obtained image will be $2.861 \cdot 144 = 412$ pixels wide and $4.181 \cdot 144 = 602$ pixels high
- With a 36 dpi scanning resolution, the obtained image will be $2.861 \cdot 36 = 103$ pixels wide and $4.181 \cdot 36 = 151$ pixels high

To make sure that

Printed image size = Acquired image size

it is necessary that

Printed image resolution = Acquired image resolution

Print and printer resolutions...

Important

Print resolution $\neq$ Printer resolution

Print resolution

Number of pixels that will be put on paper for each linear inch (usually indicated with *dpi*, although *ppi* would be more appropriate)

Printer resolution

- For inkjet printers, the number of (cyan, magenta, yellow, and black) “dots” that will be put on paper for each linear inch
- Usually, the more these dots, the better the printed image

... print and printer resolutions ...

Example

- 100 dpi print resolution → for each linear inch, 100 pixels of the image →
 $100 \times 100 = 10,000$ pixels per square inch
- 1,000 dpi printer resolution → for each linear inch, 1,000 dots → each pixel on paper will be composed of $10 \times 10 = 100$ (cyan, magenta, yellow, and black) dots, which create the pixel's color

Not considering black (to simplify), 3^{100} “colors” are possible for each pixel (actually, all possible permutations of C, M, and Y in 100 positions)

... print and printer resolutions ...

Be careful with the *printer resolution / print resolution* ratio

Example: 200 dpi print res. and 1000 dpi printer res. → The color of each pixel will be composed of $5 \times 5 = 25$ dots

For each pixel, 3^{25} possible permutations of C, M, and Y in 25 positions, instead of 3^{100} with a 100 dpi print resolution

Best printer resolution / print resolution ratio?

It depends on the quality of the printer — some trials can provide the right answer (generally, a print resolution higher than 300 dpi is useless)

Asymmetric printer resolutions

Horizontal res. $\neq$ Vertical res. (e.g., 4,800 x 2,400 dpi)

... print and printer resolutions

Example

Image taken with a digital camera, 2560 x 1920 pixels sensor (5.02 MP)

- 220 dpi print res., 2,400 x 1,200 dpi printer res. → the obtained image will be $2,560/220'' = 11.64''$ wide and $1,920/220'' = 8.73''$ high; the color of each pixel will be composed of $2,400/220 \approx 11$ dots horizontally and $1,200/220 \approx 5$ dots vertically, so ≈ 55 dots per pixel ($\approx 3^{55}$ “colors”)
- If you want to obtain a double-sized image → 110 dpi print res. → more color variations, but a coarser image
- If you want to print a high-quality, large-sized poster → many pixels are necessary!
- If you are taking a photo that will be published on the Web (72 dpi) → a small size is enough
- If you don't know in advance what you will do with your photo (and your camera has enough memory) → the maximum available size is advisable

Image file size

The image file size depends on:

1. Size in pixels

More pixels → bigger file

2. Color levels

More colors → bigger file

Example

300 x 400 px image

- 1 bit per pixel → 2 colors
 $300 \cdot 400 \cdot 1 \text{ bit} = 120,000 \text{ bits} = 15,000 \text{ byte} \approx 14.65 \text{ KB}$
- 8 bits per pixel → 256 colors
 $300 \cdot 400 \cdot 8 \text{ bits} = 960,000 \text{ bits} = 120,000 \text{ bytes} \approx 117.2 \text{ KB}$
- 8 bits per RGB channel (per pixel) → 16,777,216 colors
 $300 \cdot 400 \cdot 24 \text{ bits} = 2,880,000 \text{ bit} = 360,000 \text{ bytes} = 351.56 \text{ KB}$

Compression...

If all pixels of an image are coded, very big files are usually obtained

Compression techniques are “tricks” to reduce the file size, possibly without affecting image quality

Two simple methods to reduce the size of an image file:

1. Reduction of the image size (→ fewer pixels)

Spatial sampling rate

2. Reduction of the number of bits per pixel (i.e., the number of colors)

Quantization

But it is not always possible to reduce image size and/or number of colors...

... compression

Two basic kinds of compression:

1. *Lossless*

No information is lost with respect to the original image

2. *Lossy*

Some information is lost with respect to the original image

Lossy compressions allow higher compression levels compared to lossless compression

- Unimportant details of the image are ignored
- Acceptable when high quality levels are not strictly necessary

Some common bitmap formats...

The three “traditional” web formats (GIF, JPEG, and PNG), and others, will be analyzed in the following slides

BMP (BitMaP)

- Created by Microsoft and IBM (DOS, OS/2, and Windows operating systems)
- Supports 24-bit images (8 bits per channel)
- Not compressed

TIFF (Tagged Image File Format)

- Used to be the standard for scanned images
- Independent of the platform (PC, Mac, etc.)
- 8/16 bits per channel
- Can be compressed

... some common bitmap formats...

PCX

Created by Zsoft for Paintbrush (old software released in 1985)

TGA (TrueVision TarGA)

Created by Truevision for early graphic cards supporting true color. No transparency, but can use the *alpha* channel

PGM (Portable GrayMap), PPM (Portable PixelMap), PBM (Portable BitMap)

UNIX native formats for gray-level (PGM) and color (PPM and PBM) images

... some common bitmap formats...

PCD (Kodak Photo CD)

- Created in 1992 by Kodak, used to store photos on CDs
- Incorporates different image sizes

PSD

Adobe Photoshop

XCF

GIMP

CPT

Corel Photo-Paint

PSP

PaintShop Pro

... some common bitmap formats

Raw formats

- Also called “digital negatives”
- Specific to the camera brand
 - NEF (Nikon)
 - CR2 (Canon)
 - ORF (Olympus)
 - PEF (Pentax)
 - SR2 (Sony)
 - RW2 (Panasonic)
 - ...
- Usually, no in-camera processing for white balance, hue, tone, and sharpening are applied
- Usually based on the TIFF format

GIF format...

GIF = Graphic Interchange Format

Created in 1987 by *Compuserve Information Service* and adopted as a web format because of its efficiency

Features

- Supported by all browsers (even the oldest ones)
- 8 bits → 256 colors (at most, but they can also use less colors)
- Colors of the palette coded with 24 bits
- Compressed → (usually) small files
- Different versions (GIF87a, GIF89a)

... GIF format

GIF files are compressed

Lossless compression (*LZW* - Lempel-Ziv-Welch technique)

- No loss of information – Higher compression with images containing large single-color areas
- Sequences of same-color pixels on a row are coded as a couple *number of pixels* - *color*

True color → GIF conversion: strong color reduction



Interlaced GIF images

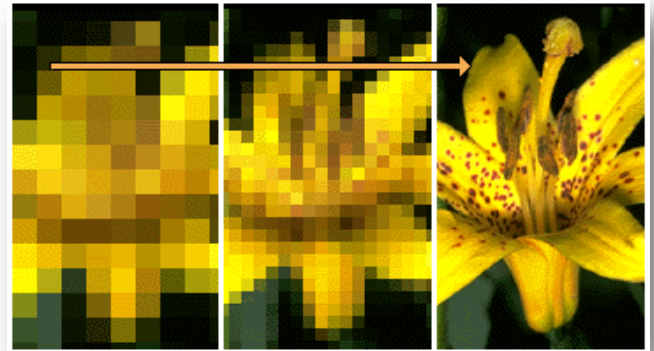
Ordinary GIF images are displayed by the web browser “at once”

Nothing is displayed until the file has been completely downloaded

Interlaced GIFs are displayed “in pieces”

From low to high resolution.

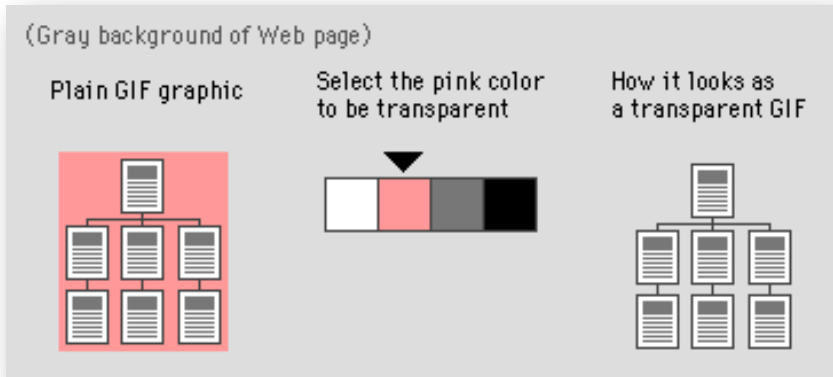
The user can “get an idea” of the image content since the beginning



GIF images - Transparency

The GIF89a format allows a color of the CLUT of the image to become transparent

E.g., the background



GIF images - Dithering

The conversion of a true color image into the GIF format causes a loss in color information

However, tools like Photoshop or GIMP reduce image deterioration through the *dithering* process: certain pixels in the CLUT are arranged close together to “simulate” missing colors



GIF images - Palette

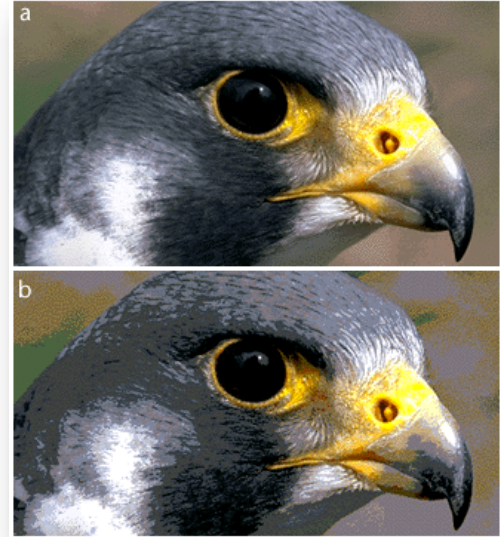
A conversion software selects the 256 colors of the CLUT that best represent the image to be converted into a GIF

Different images will (usually) have different palettes

Two images can even use 512 different colors

What if we are using an old computer that can only display 256 colors at a time (8 bits/pixel)?

Colors in the two images will be distorted...

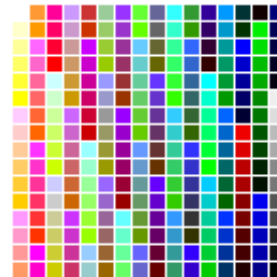


GIF images - System Palette...

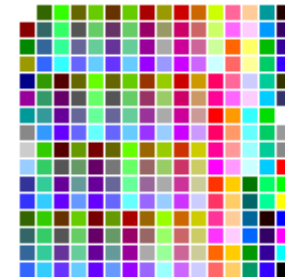
Web browsers solve(d) the problem of simultaneously displaying colors of multiple GIF images on computers with only 256 colors by using a special palette, called the *system palette*

- All colors are taken from the system palette
- Different for Windows and Mac

Macintosh system palette

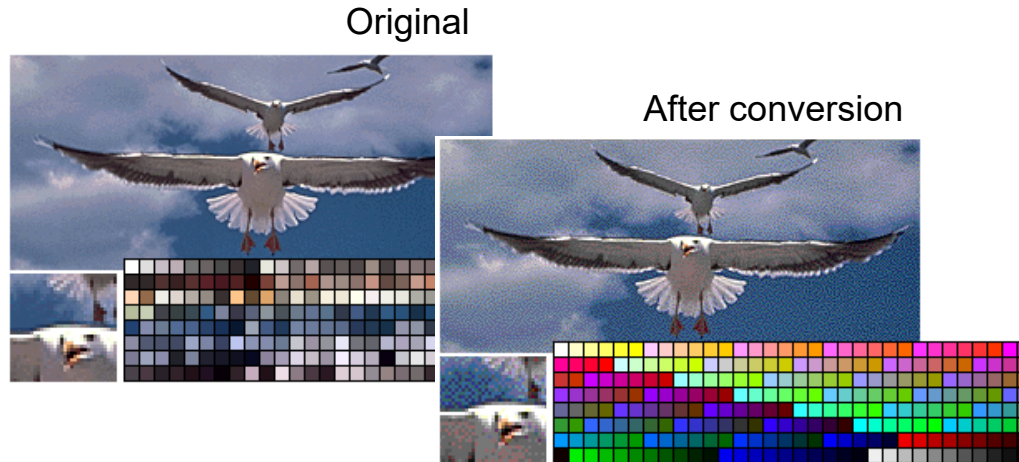


Windows system palette



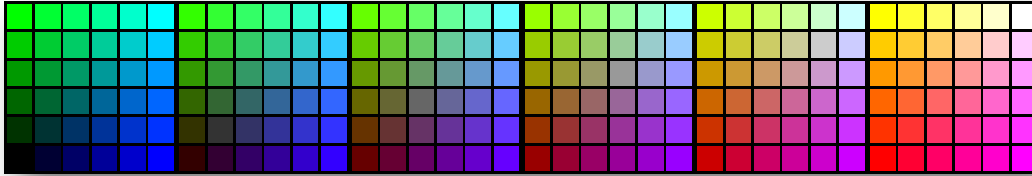
... GIF images - System Palette

Example of conversion from custom palette to Macintosh system palette



GIF images - Safe Palette

The system palettes of Windows and Mac have 216 colors in common



- These 216 colors form the *safe palette*
- By using only colors from the safe palette (for example, when creating a website), we could be sure that they would display exactly the same on both Windows and Mac systems

GIF images and palette - Conclusions

The problem of images using different palettes is now practically insignificant

Almost any computer or device supports millions (24 bits) of colors

Each image can have its own palette (*custom palette*)

But the concept of system palette survives (e.g., *Web Colors* in Photoshop and GIMP)

Important: always keep a copy of the original, actual color image before converting it

Otherwise, color information is definitely lost!

JPEG format...

JPEG = Joint Photographic Experts Group

Format suitable for photos and paintings

Features

- True color (24 bits)
- Compressed, with selectable compression levels: higher compression → lower quality
- Supported by (almost) all browsers (not the oldest ones)
- There exists an interlaced version called *progressive JPEG*

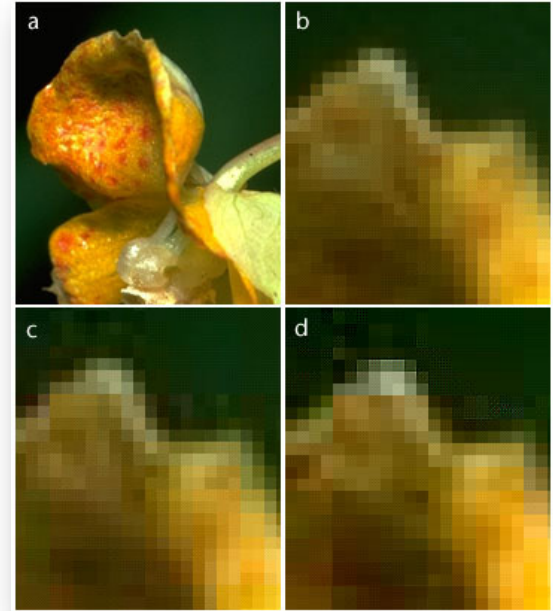
... JPEG format...

The compression level is selected just before saving the file

File size can be significantly reduced with respect to the original image file

JPEG compression can be both lossy and lossless (JPEG 2000)

In the lossy version (the most common), “unimportant” details are removed → loss of information

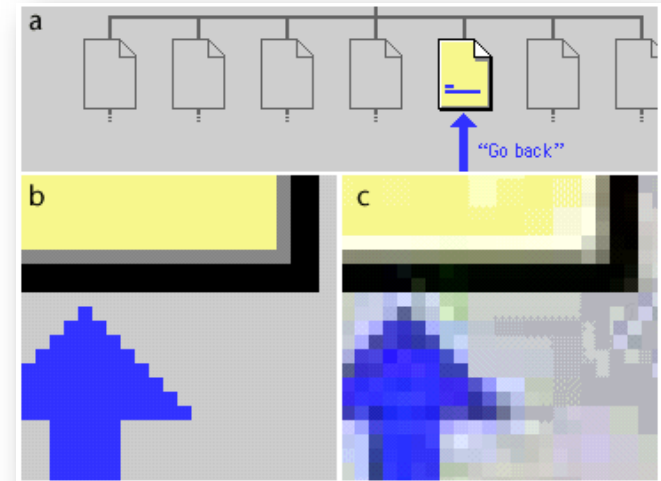


... JPEG format

The JPEG algorithm works better with “real” images (photos, paintings, etc.)

... smooth color transitions

Generally, **not suitable** for drawings or pictures with sharp edges and abrupt color transitions



PNG format

PNG = Portable Network Graphics

- Format created explicitly for the Web (supported by versions 4.x of browsers)
- Standardized by the World Wide Web Consortium (W3C) (in 1994, Unisys tried to claim the rights for the LZW compression algorithm of the GIF format)
- Lossless
- Supports gray-level (16 bits) and color (48 bits) images
- Supports sophisticated “effects” (e.g., *alpha* channels for gradual transparency and creation of masks)

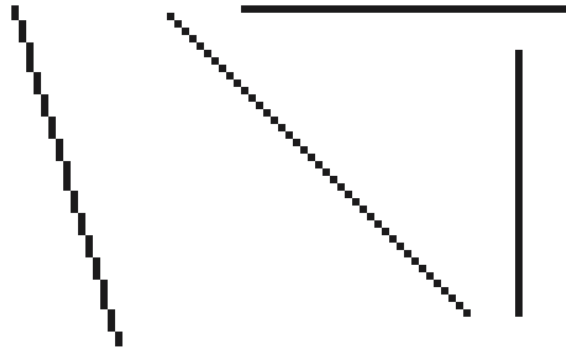
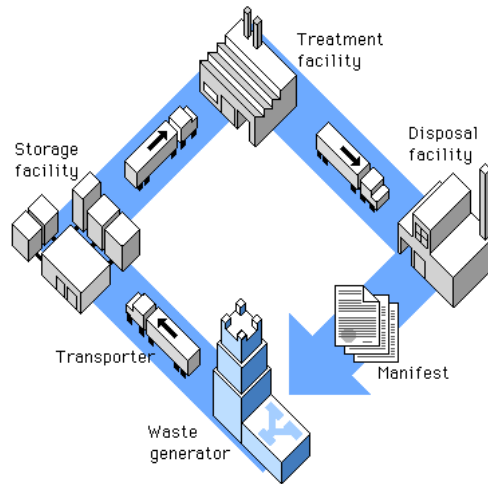
Recent Web graphic formats

- **APNG (Animated Portable Network Graphics)**
For lossless animation sequences (GIF is less performant)
- **AVIF (AV1 Image File Format)**
For both images and animations, it offers much better compression than PNG or JPEG, with support for higher color depths
- **WebP (Web Picture format)**
For both images and animations, it offers much better compression than PNG or JPEG, with support for higher color depths

Drawings and diagrams

In drawings and diagrams, only vertical, horizontal, and 45° lines are not “jagged”

annoying effect...



Antialiasing

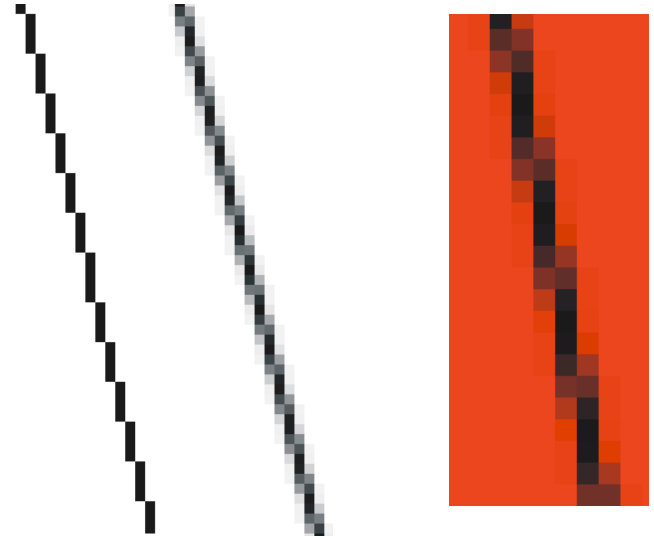
Allows “jagged effects” to be reduced

Adds new colors by smoothing color transitions near the line

Depends on the background color

Supported by almost any photo editing software

E.g., Photoshop and GIMP



Vector graphics...

Image elements are described mathematically

E.g., segments, circles, ellipses, rectangles, curves, etc.

For each element, the parameters necessary for its description are stored in the graphic file

E.g., for a segment: x and y coordinates of the start and end points, thickness, color;
for a circle: the coordinates of its center, radius, border thickness and color, filling color

Vector graphics is also called *object-oriented graphics*

Because images are considered as sets of “objects”

... vector graphics

Drawings can be resized without loss of quality

Original image



Enlargement -
vector



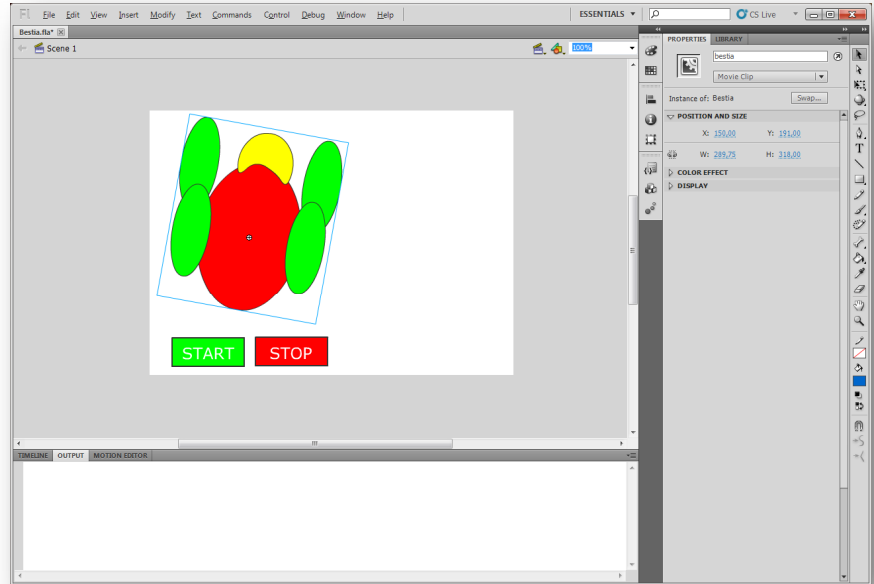
Enlargement -
bitmap



Software for vector graphics

Drawings are treated as sets of objects

A widespread software is Adobe Illustrator

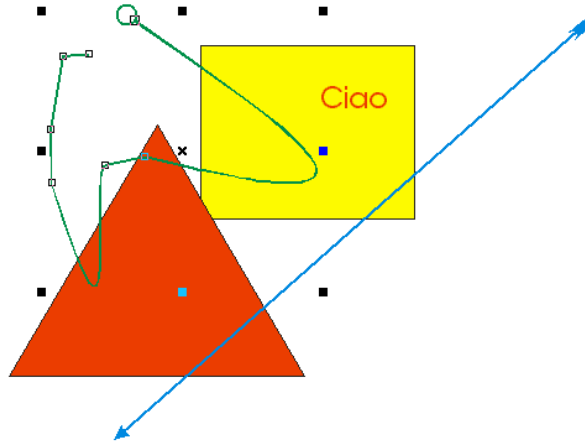


Operations

Many tools for acting on vector images are similar to those used in photo editing software (such as Photoshop)

For example:

- Selection
- Zoom
- Creation of shapes
- Text
- ...



Most common vector formats

WMF (Windows Meta File)

Native Windows vector format

AI

Adobe Illustrator

DWG

Autocad

CDR

CorelDraw

SWF (old)

Adobe Flash

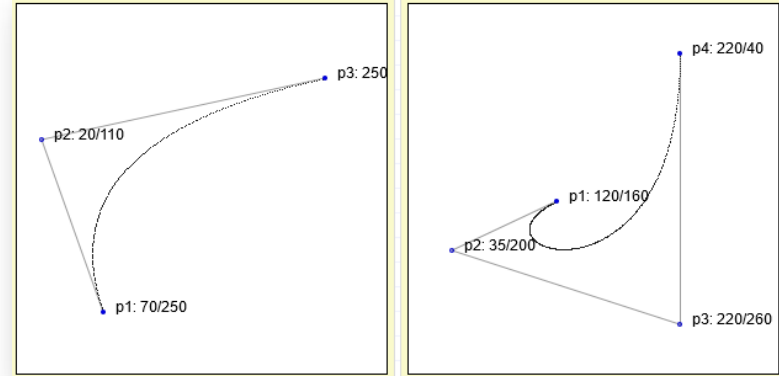
SVG (Scalable Vector Graphics)

The “standard” vector format for the Web



Bézier curves

Are interpolation functions: given a set of points, they generate “values” between those points



To change a curve, it is necessary to change the weights of each point, changing the interpolation